

NutriQuest — full logic breakdown

GLOBAL IMPLEMENTATION / SOURCE-OF-TRUTH RULES

- This document is the new authoritative product/system specification for the NutriQuest codebase and should be treated as the source of truth for feature behavior, logic, formulas, progression, rewards, and implementation decisions.
- A separate task list will also be provided and should be used alongside this document for implementation sequencing and specific work items.
- Existing code, old notes, legacy screens, or previously implemented behavior that conflict with this document should be updated to match this document unless a later task explicitly overrides it.
- Some finalized features in this document may already have complete frontend/UI, may have only partial frontend support, or may currently exist only in backend logic.
- For every finalized feature, first reuse and adapt any suitable existing NutriQuest UI/UX when available rather than unnecessarily rebuilding it.
- If a finalized feature has no frontend/UI/UX connected to it, create the required screens, components, interactions, states, and navigation needed for the feature to be usable by the player.
- Any newly created UI/UX must remain visually and behaviorally consistent with NutriQuest's overall existing design language, including its layout style, typography, spacing, controls, cards, navigation patterns, game theming, feedback states, and responsive behavior.
- Do not omit a finalized feature simply because a UI was not explicitly designed during this planning document. Missing frontend should be treated as implementation work, not as a reason to skip the feature.

1. FOOD → CHARACTER / NUTRITION SCAN PIPELINE

Two food-input paths exist, but only one can create a monster.

Barcode scan — character-generating input

- The user scans a packaged-food barcode.
- The barcode is validated and used to look up the product's nutrition data through Open Food Facts or an equivalent nutrition source.
- The full nutritional profile is evaluated as a combined input. Individual macros are NOT permanently mapped to specific battle stats.
- A model uses the overall nutrition profile together to determine the generated monster's core attributes.
- Barcode scanning is the only way a user can directly input food and generate a character.
- Characters can also be obtained from cases/crates, but those are game-generated rewards rather than user food inputs.

Real-food / dish photo analysis — nutrition only

- A user can take a photo of real food or a prepared dish for nutrition analysis.
- The vision/nutrition pipeline can identify the meal, estimate portions, calories, macros, and other relevant nutritional information.
- Real-food or dish-photo analysis NEVER creates or mints a monster.

- This path exists only to provide useful nutrition feedback and tracking information to the user.

Monster stats generated from barcode nutrition

- Health
- Base Attack
- Rarity
- Starting Stars — fixed at ★1 for all barcode-generated monsters
- Mana — only present on Epic-rarity monsters and above.

Mana / special abilities

- Epic, Legendary, Mythic, Secret, and any future rarity above Epic can have Mana.
- Mana exists because these higher-rarity characters can use special abilities.
- Monsters below Epic do not use Mana and do not have these special-ability mechanics.

Stat-generation principle

- Nutrition should be treated holistically rather than with rules such as protein = attack or fiber = defense.
- The full combination of calories, macros, micronutrients, ingredient/nutrition quality, and other available nutrition signals is fed into the model.
- The model determines an appropriate Health, Base Attack, and Rarity profile for the monster. Starting Stars are not model-generated; barcode monsters always begin at ★1.
- Exact model weighting and balancing can be tuned separately without changing the user-facing scan flow.

2. CHARACTERS / MONSTERS

Monster generation and identity

- The current demo has 14 authored character designs loaded into the game. Barcode scans generate a new monster instance using the scanned product's nutrition data; cases/crates generate game-reward monster instances.
- Barcode-generated monsters always begin at ★1.
- Every monster uses Health, Base Attack, Rarity, Stars, and Net Worth. Epic rarity and above can additionally use Mana/special abilities.
- Power Bands are removed completely. Shiny variants are excluded from the current system and may be revisited later as an optional cosmetic/collectible feature with no required combat advantage.

Rarity tiers and combat multipliers

- Common ×1.00
- Uncommon ×1.06
- Rare ×1.12
- Epic ×1.25
- Legendary ×1.40
- Mythic ×1.55

- Secret $\times 1.70$

Nutrition score used for barcode rarity generation

- Barcode nutrition is normalized per 100 g / 100 mL where possible.
- ProteinScore = $\text{clamp}(\text{protein_g} / 25, 0, 1) \times 100$.
- FiberScore = $\text{clamp}(\text{fiber_g} / 10, 0, 1) \times 100$.
- SugarScore = $(1 - \text{clamp}(\text{sugar_g} / 20, 0, 1)) \times 100$.
- SodiumScore = $(1 - \text{clamp}((\text{sodium_mg} - 50) / 950, 0, 1)) \times 100$.
- SatFatScore = $(1 - \text{clamp}((\text{satFat_g} - 0.5) / 9.5, 0, 1)) \times 100$.
- NutritionScore = $0.25 \times \text{ProteinScore} + 0.20 \times \text{FiberScore} + 0.25 \times \text{SugarScore} + 0.15 \times \text{SodiumScore} + 0.15 \times \text{SatFatScore}$.
- If a nutrition field is unavailable, its weight is redistributed proportionally across the available components rather than treating missing data as zero.
- Nutrition affects rarity probability rather than guaranteeing a rarity. No single macro can determine rarity by itself.

Nutrition-influenced rarity roll

- Baseline odds at NutritionScore 50: Common 80%, Uncommon 16%, Rare 3.2%, Epic 0.64%, Legendary 0.128%, Mythic 0.0256%, Secret 0.0064%.
- Rarity indexes: Common=0, Uncommon=1, Rare=2, Epic=3, Legendary=4, Mythic=5, Secret=6.
- For each rarity r : $\text{rawWeight}(r) = \text{baseOdds}(r) \times \exp(0.8 \times ((\text{NutritionScore} - 50) / 50) \times \text{rarityIndex}(r))$.
- Normalize all seven raw weights so the final probabilities sum to 100%.
- Approximate reference distributions:
 - Score 0: Common 91.01%, Uncommon 8.18%, Rare 0.74%, Epic 0.066%, Legendary 0.006%, Mythic 0.001%, Secret $\sim 0.0001\%$.
 - Score 25: Common 86.59%, Uncommon 11.61%, Rare 1.56%, Epic 0.209%, Legendary 0.028%, Mythic 0.004%, Secret 0.001%.
 - Score 50: Common 80.00%, Uncommon 16.00%, Rare 3.20%, Epic 0.640%, Legendary 0.128%, Mythic 0.026%, Secret 0.006%.
 - Score 75: Common 70.17%, Uncommon 20.94%, Rare 6.25%, Epic 1.864%, Legendary 0.556%, Mythic 0.166%, Secret 0.062%.
 - Score 100: Common 55.62%, Uncommon 24.76%, Rare 11.02%, Epic 4.905%, Legendary 2.183%, Mythic 0.972%, Secret 0.541%.

Combat-profile generation from nutrition

- NutritionScore does not directly become Health or Base Attack; the raw nutrition profile is evaluated again so rarity quality and combat profile are not the same calculation.
- AttackProfile = 40% normalized protein + 25% normalized carbohydrate energy + 20% normalized calorie density + 15% normalized fat energy. Inputs are capped/normalized so no single nutrient can create extreme Attack by itself.
- HealthProfile = 25% normalized fiber + 20% normalized protein + 20% sugar quality + 20% sodium quality + 15% saturated-fat quality.

- These profiles determine the relative generated Health/Base Attack distribution. Final battle-scale constants are resolved with the Pokémon-style battle system.

Fresh ★1 net-worth bands

- Common: 500–1,099
- Uncommon: 1,100–2,649
- Rare: 2,650–6,899
- Epic: 6,900–19,499
- Legendary: 19,500–58,499
- Mythic: 58,500–186,999
- Secret: 187,000–597,999
- Rarity is determined first; rarity then determines the legal net-worth band. A fresh ★1 mint never crosses its rarity band.

Weighted mint-value roll

- Once rarity is known, a hidden weighted roll chooses the monster's base mint value inside that rarity's band.
- Bottom 40% of band: 55% chance.
- 40–70% of band: 27% chance.
- 70–90% of band: 13% chance.
- 90–98% of band: 4% chance.
- Top 2% of band: 1% chance.
- After a band segment is selected, the value is rolled uniformly inside that segment.
- The resulting baseMintValue is permanent for that monster lineage. Net Worth is an economy/reward value and is NOT a direct proxy for combat strength.

Stars / mastery

- Common through Mythic can progress ★1 → ★5. Secret is capped at ★2 to prevent terminal-rarity combat scaling from becoming effectively unbeatable.
- Fusion requires 3 monsters with the same character, same rarity, and same star level. Rarity never changes through fusion.
- Copy-equivalent mastery cost: ★2 = 3 ★1 copies, ★3 = 9, ★4 = 27, ★5 = 81.
- Stars increase Health and Base Attack through combat mastery scaling. Current v1 combat multipliers: ★1 ×1.00, ★2 ×1.08, ★3 ×1.18, ★4 ×1.30, ★5 ×1.45. Secret can only use ★1/★2.
- Rarity and star combat scaling are multiplicative with each other; Net Worth star bonuses remain additive and are never multiplied.

Fusion value inheritance

- Fusion automatically preserves the highest underlying baseMintValue among the 3 consumed monsters. The player does not choose a primary monster.
- Example: ★1 Common values 540, 810, and 1,050 fuse into a ★2 whose inherited baseMintValue is 1,050; the ★2 Common bonus is then added.
- This creates a deliberate progression choice: merge immediately for faster mastery or wait for a stronger mint-value roll to use as the lineage foundation.

Additive star bonuses to Net Worth

- $\text{CurrentNetWorth} = \text{baseMintValue} + \text{starBonus}[\text{rarity}][\text{stars}]$.
- Common: ★1 +0, ★2 +90, ★3 +240, ★4 +450, ★5 +833.
- Uncommon: ★1 +0, ★2 +233, ★3 +620, ★4 +1,163, ★5 +2,188.
- Rare: ★1 +0, ★2 +638, ★3 +1,700, ★4 +3,188, ★5 +6,140.
- Epic: ★1 +0, ★2 +1,890, ★3 +5,040, ★4 +9,450, ★5 +18,450.
- Legendary: ★1 +0, ★2 +5,850, ★3 +15,600, ★4 +29,250, ★5 +58,275.
- Mythic: ★1 +0, ★2 +19,275, ★3 +51,400, ★4 +96,375, ★5 +190,150.
- Secret: ★1 +0, ★2 +61,650 only.
- For Common through Mythic, ★2/★3/★4 bonuses are 15%/40%/75% of the rarity band step. ★5 is anchored so the bottom-value ★5 monster reaches the bottom-value ★2 monster of the next rarity: $\text{bonus5}(N) = \text{step}(N) + \text{bonus2}(N+1)$.
- Mastery may push Current Net Worth above the monster's original ★1 rarity band. This is intentional and represents accumulated investment; it does not change the monster's rarity.

Design principle

- Rarity = natural quality/potential and is nutrition-influenced for barcode generation.
- Base mint roll = the individual monster's economic quality inside its rarity.
- Stars = player investment/mastery.
- Net Worth = rarity band + individual mint roll + additive mastery bonus.
- Combat strength = nutrition-generated Health/Attack × rarity scaling × star mastery, plus moves/abilities; it is not calculated from Net Worth.

3. COOKBOOKS / CHARACTER ACQUISITION

Acquisition flow

- Spendable Coins → Cookbook → Rarity Case → Monster.
 - Cookbooks are the purchasable random-reward container. Cases represent the rarity reward won from a Cookbook.
 - The seven Cases are Common, Uncommon, Rare, Epic, Legendary, Mythic, and Secret.
- Opening a Case produces a monster of exactly that rarity.
- Cookbook monsters begin at ★1 and use the same weighted base-mint-value system defined in the Character System.
 - There is NO pity system. Every Cookbook opening is an independent roll using its published fixed odds. No rarity is guaranteed after any number of misses.

Currency and selling

- Cookbooks are purchased with Coins; Keys are removed from this acquisition system.
- Monster Net Worth is denominated directly in Coins and is the monster's actual sell value.
- Selling a monster permanently removes it from inventory and adds Current Net Worth to the player's spendable Coin balance.
- Coin Balance and Monster Net Worth are separate while the monster is owned; Net Worth only becomes spendable Coins when the monster is sold.

Cookbook tiers, prices, and Case odds

- Home Cookbook — price 🍲 1,600: Common 80%, Uncommon 16%, Rare 3.2%, Epic 0.64%, Legendary 0.128%, Mythic 0.0256%, Secret 0.0064%.
- Chef's Cookbook — price 🍲 3,300: Common 55%, Uncommon 28%, Rare 12%, Epic 4%, Legendary 0.8%, Mythic 0.18%, Secret 0.02%.
- Master Cookbook — price 🍲 9,200: Common 20%, Uncommon 35%, Rare 25%, Epic 14%, Legendary 4.8%, Mythic 1.1%, Secret 0.1%.
- Forbidden Cookbook — price 🍲 33,500: Common 0%, Uncommon 10%, Rare 25%, Epic 35%, Legendary 20%, Mythic 9.5%, Secret 0.5%.

Expected-value balancing

- Approximate expected fresh ★1 sell value by rarity under the existing 55%/27%/13%/4%/1% weighted mint roll: Common 🍲 746; Uncommon 🍲 1,735; Rare 🍲 4,392; Epic 🍲 12,066; Legendary 🍲 35,490; Mythic 🍲 111,185; Secret 🍲 355,510.
- $\text{CookbookEV} = \sum [P(\text{rarity}) \times \text{ExpectedNetWorth}(\text{rarity})]$.
- Approximate Cookbook EVs: Home 🍲 1,188; Chef's 🍲 2,461; Master 🍲 6,826; Forbidden 🍲 24,932.
- Cookbook prices are set near $\text{EV} / 0.745$, producing an average sell-back return of roughly 74–75%: Home 74.3%, Chef's 74.6%, Master 74.2%, Forbidden 74.4%.
- This deliberately makes repeated Buy → Open → Sell negative expected value and prevents an infinite-coin loop while preserving high-upside rare pulls.

Fairness / roll integrity

- Cookbook rarity rolls remain server-authoritative. The existing cryptographic commit-reveal approach can be retained: $\text{roll} = \text{HMAC_SHA256}(\text{serverSeed}, \text{clientSeed}:\text{nonce}:\text{cursor})$, with the seed hash committed before use and the seed revealed on rotation so historical rolls can be independently verified.
- Fixed published odds plus independent rolls make the statistical model transparent and keep expected-value calculations stable.

Presentation / technical math to preserve

- Seven-rarity probability distributions for each of the four Cookbooks.
- Expected-value equation: $\text{EV} = \sum [P(\text{rarity}) \times E(\text{NetWorth} | \text{rarity})]$.
- Weighted mint distribution: 55% / 27% / 13% / 4% / 1% across increasing percentile segments of each rarity band.
- Cookbook pricing target: $\text{Price} \approx \text{EV} / 0.745$.
- Expected sell-back rate: $\text{EV} / \text{Price} \approx 74\text{--}75\%$.
- HMAC-SHA256 commit-reveal formula for verifiable random rolls.

4. BATTLE SYSTEM

Core battle format

- Battles use 3-monster teams with one active monster per side at a time.

- On a turn, the player can use an attack, use an eligible Special Ability, or voluntarily switch monsters.

- A monster faints at 0 HP. The battle ends when all 3 monsters on one side have fainted.

- There is no separate monster-level system. Combat progression comes from Base Attack/Health, rarity, stars, moves, and abilities.

Fainting and recovery

- Monsters that survive a normal battle immediately return to full Health after the battle.

- Monsters that reach 0 HP remain fainted and cannot be used until recovered.

- A fainted monster can recover early by completing an eligible health/nutrition task. The exact recovery-task pool is defined later in the Nutrition / Health Layer.

- Otherwise, fainted monsters automatically recover at the next daily reset. This is a fixed daily reset rather than an individual 24-hour timer from the moment of fainting.

- Forfeiting does not remove fainting consequences: any monster that already reached 0 HP remains fainted.

- Special modes such as a future Dungeon may override normal post-battle healing rules and use persistent HP if explicitly designed that way.

Turn order

- PvP: the server performs a fair 50/50 coin flip at the beginning of the battle to determine who takes the first turn. $P(\text{Player A starts})=0.50$ and $P(\text{Player B starts})=0.50$.

- PvE / AI: the human player always takes the first turn.

- After the first turn, turns alternate normally. There is no Speed, Tempo, initiative, or turn-order stat.

Moves and character identity

- Every monster has 3 authored standard moves.

- Epic, Legendary, Mythic, and Secret monsters additionally have 1 authored Mana-based Special Ability.

- Standard moves do not consume Mana.

- Each character has a unique authored moveset tailored to that character. NutriQuest uses Pokémon-style battle concepts as a mechanical reference, but move names, status names, visuals, values, and effects are tailored to NutriQuest.

- Moves may deal direct damage, combine damage with a secondary status effect, apply a status without damage, buff the user, debuff the opponent, heal, or provide other character-specific utility.

- Common move fields can include Power, Accuracy, StatusEffect, StatusChance, Duration, and ManaCost where applicable.

- Stronger moves can be balanced through lower accuracy, drawbacks, Mana cost, or weaker secondary effects rather than simply making every move stronger.

Status effects

- Status mechanics can use concepts similar to Pokémon—damage over time, disabling, turn disruption, Attack reduction, accuracy reduction, healing, buffs, debuffs, and similar effects—but NutriQuest statuses are individually authored and themed for its characters.

- Status effects do not need to share one rigid global behavior merely because they use a similar underlying mechanic.

- For a secondary status that only occurs after a successful hit: $P(\text{Status Applied per attempt}) = P(\text{Hit}) \times P(\text{Status} \mid \text{Hit})$.

- Example: 90% move accuracy with a 20% status chance gives $0.90 \times 0.20 = 18\%$ status probability per attempted use.

Combat scaling and damage

- Rarity and star combat multipliers from the Character System remain the source of progression scaling.

- $\text{EffectiveAttack} = \text{BaseAttack} \times \text{RarityMultiplier} \times \text{StarMultiplier}$.

- Health is scaled using the same locked rarity/star combat progression principle.

- Pokémon is the reference for attack/damage behavior, but the formula is adapted because NutriQuest intentionally has no monster Level or Defense stat.

- $\text{BaseDamage} = ((\text{MovePower} \times \text{EffectiveAttack}) / \text{DamageScale}) + 2$.

- $\text{FinalDamage} = \text{floor}(\text{BaseDamage} \times \text{CriticalModifier} \times \text{RandomVariance} \times \text{OtherMoveModifiers})$.

- DamageScale is a global balancing constant to be tuned against the actual Health/Base Attack ranges of the authored characters rather than guessed before playtesting.

- Successful damaging attacks use a random damage variance uniformly distributed from 0.85 to 1.00.

- Base critical-hit chance follows the modern Pokémon-style reference: $P(\text{Critical}) = 1/24 \approx 4.17\%$, with $\text{CriticalModifier} = 1.5\times$. Individual authored moves may modify critical behavior.

- Accuracy is move-specific: $P(\text{Hit}) = \text{MoveAccuracy}/100$. A miss deals 0 damage and normally does not apply a hit-dependent secondary status unless that move explicitly defines different behavior.

- Net Worth never affects combat damage or combat strength.

No type system

- The old Protein/Fiber/Hydration/Vitamin element system is removed completely.

- NutriQuest has no type weaknesses, resistances, STAB, type triangle, element squad bonus, or type-effectiveness multiplier.

- Battle strategy instead comes from unique movesets, statuses, Mana Specials, switching, team composition, rarity, and stars.

Mana and Special Abilities

- Mana exists only for Epic rarity and above.

- Base Mana is authored per character according to that character's Special Ability and intended playstyle; it is not nutrition-derived.

- Rarity does not multiply Mana.

- $\text{StartingMana} = \text{floor}(\text{BaseMana} \times \text{ManaStarMultiplier})$.
- Mana star multipliers: ★1 $\times 1.00$, ★2 $\times 1.10$, ★3 $\times 1.20$, ★4 $\times 1.35$, ★5 $\times 1.50$. Secret remains limited to its allowed ★1/★2 progression.
- Monsters enter each battle with full Starting Mana.
- Mana does not naturally regenerate during battle.
- Character-specific moves/statuses may restore, drain, steal, or otherwise manipulate Mana if explicitly authored to do so.
- Special Abilities consume their authored ManaCost. Higher-impact Specials can therefore be balanced with higher Mana costs.

Switching

- A voluntary switch consumes the player's entire turn.
- When an active monster faints, the player chooses a remaining non-fainted monster to enter. This forced replacement does not consume the player's next turn.
- Fainted monsters cannot be switched back into battle.

Ranked / matchmaking direction

- Player ranked progression is separate from monster progression.
- The competitive ladder is Iron $\rightarrow$ Bronze $\rightarrow$ Silver $\rightarrow$ Gold $\rightarrow$ Platinum $\rightarrow$ Diamond.
- Ranked wins increase RR (Rank Rating) and ranked losses decrease RR.
- A separate set of eligible tasks can award small amounts of positive RR, but battle performance remains the main ranked-progression source.
- Skill-based matchmaking (SBMM) will be used so players are not routinely matched into severely mismatched teams, such as lower-strength teams repeatedly facing full Epic+ squads.
- Exact RR gains/losses, rank thresholds, task RR limits, hidden matchmaking rating, and team-strength weighting are intentionally deferred to System 5 / Ranked & Matchmaking design.

Presentation / technical math to preserve

- PvP starting-turn probability: $P(A \text{ starts}) = P(B \text{ starts}) = 0.50$.
- $\text{EffectiveAttack} = \text{BaseAttack} \times \text{RarityMultiplier} \times \text{StarMultiplier}$.
- $\text{BaseDamage} = ((\text{MovePower} \times \text{EffectiveAttack}) / \text{DamageScale}) + 2$.
- $\text{FinalDamage} = \text{floor}(\text{BaseDamage} \times \text{CriticalModifier} \times \text{RandomVariance} \times \text{OtherMoveModifiers})$.
- $\text{RandomVariance} \sim \text{Uniform}(0.85, 1.00)$.
- $P(\text{Critical}) = 1/24 \approx 4.17\%$; $\text{CriticalModifier} = 1.5\times$.
- $P(\text{Hit}) = \text{MoveAccuracy}/100$.
- $P(\text{Status Applied}) = P(\text{Hit}) \times P(\text{Status} | \text{Hit})$ for hit-dependent secondary effects.
- $\text{StartingMana} = \text{floor}(\text{BaseMana} \times \text{ManaStarMultiplier})$, using ★1 1.00 / ★2 1.10 / ★3 1.20 / ★4 1.35 / ★5 1.50.
- Preserve the distinction between combat progression (Health/Attack, rarity, stars, moves, Mana) and economic value (Net Worth).

5. RANKED / MATCHMAKING / GAME MODES

Rank ladder

- Ranked progression is separate from monster progression.
- Ranks: Iron 0–99 RR → Bronze 100–199 → Silver 200–299 → Gold 300–399 → Platinum 400–499 → Diamond 500+.
- No sub-divisions and no promotion matches for the hackathon build.

Ranked RR

- Base ranked win = +20 RR. Base ranked loss = -15 RR.
- $RRDifference = OpponentRR - PlayerRR$.
- $Adjustment = clamp(round(RRDifference / 25), -5, +5)$.
- $WinRR = 20 + Adjustment$, giving a practical range of +15 to +25 RR.
- $LossRR = -15 + Adjustment$, giving a practical range of -20 to -10 RR.
- Stronger opponents therefore give more RR when beaten and cost less RR when lost to; weaker opponents do the reverse.

Task RR

- Eligible ranked/nutrition tasks award +5 RR, capped at +10 task RR per day.
- Task RR cannot promote a player into the next rank. It can only bring the player to the top of the current rank; the promotion must come from a ranked battle win.

Simple SBMM

- Matchmaking considers both player RR and the combat strength of the selected 3-monster team.
- $MonsterPower = EffectiveHealth + (EffectiveAttack \times AttackWeight)$.
- $TeamPower = \sum MonsterPower$ for the 3 selected monsters.
- $TeamGap\% = |TeamPowerA - TeamPowerB| / \max(TeamPowerA, TeamPowerB)$.
- $MatchScore = (|RRA - RRB| / 100) + TeamGap\%$.
- Lower MatchScore = better/preferred opponent.
- AttackWeight is a tuning constant to be set once actual authored Health/Attack ranges are finalized.
- Net Worth is not used for matchmaking. Rarity/stars are already represented through effective combat stats and are not double-counted separately.

Ranked win Case reward

- A ranked win gives both the calculated RR gain and exactly 1 rank-based Rarity Case roll. A ranked loss gives the RR loss and no Case.
- Backend flow: Ranked Win → RR Calculation → $weightedRandom(RankedCaseOdds[current\ rank])$ → award existing Rarity Case.
- The awarded Case uses the existing System 3 Case logic; this is not a Cookbook opening and does not create a separate monster-generation system.
- Every ranked Case roll is independent. There is no pity system.
- Ranked Case odds by current rank:
 - Iron: Common 75%, Uncommon 18%, Rare 5%, Epic 1.6%, Legendary 0.32%, Mythic 0.07%, Secret 0.01%.

Bronze: Common 65%, Uncommon 23%, Rare 8%, Epic 3%, Legendary 0.8%, Mythic 0.18%, Secret 0.02%.

Silver: Common 52%, Uncommon 28%, Rare 13%, Epic 5%, Legendary 1.6%, Mythic 0.35%, Secret 0.05%.

Gold: Common 38%, Uncommon 30%, Rare 20%, Epic 8.5%, Legendary 2.7%, Mythic 0.7%, Secret 0.1%.

Platinum: Common 24%, Uncommon 28%, Rare 27%, Epic 14%, Legendary 5.5%, Mythic 1.3%, Secret 0.2%.

Diamond: Common 15%, Uncommon 25%, Rare 30%, Epic 20%, Legendary 7.5%, Mythic 2.25%, Secret 0.25%.

- Approximate Epic+ chance: Iron 2%, Bronze 4%, Silver 7%, Gold 12%, Platinum 21%, Diamond 30%.

- Approximate expected fresh ★1 monster value per ranked win under the existing mint-value model: Iron ~1.5k Coins, Bronze ~2.1k, Silver ~3.2k, Gold ~4.8k, Platinum ~7.5k, Diamond ~10.3k.

- Ranked reward EV uses: $\text{ExpectedRewardValue}(\text{rank}) = \sum [P(\text{case rarity} \mid \text{rank}) \times E(\text{NetWorth} \mid \text{rarity})]$.

Game modes

- Ranked PvP: 3v3 competitive battle, uses SBMM, modifies RR, and awards a rank-based Case on a win.

- Friendly PvP: direct/LAN 3v3 battle using the same core combat rules with no RR change and no ranked Case reward.

- Dungeon: the primary PvE mode. The separate standalone AI Battle, old Async Challenge, and old Arena wager mode are removed.

Dungeon rules

- Player enters with exactly 3 monsters and progresses through endless AI-controlled floors. There is no final floor; the objective is to reach the highest floor possible.

- $\text{DungeonScore} = \text{HighestFloorReached}$; personal best can be stored and can support a future Dungeon leaderboard.

- HP persists between Dungeon floors instead of restoring after each fight.

- If a monster reaches 0 HP, it is unavailable for the remainder of that Dungeon run. The player continues $3 \rightarrow 2 \rightarrow 1$ monsters as the squad is reduced. No replacement from inventory is allowed mid-run.

- The run ends when all 3 monsters have reached 0 HP.

- Dungeon fainting is run-only. At the end of the run, Dungeon HP/fainted state is cleared and the monsters return to their normal usable state; Dungeon fainting does not trigger the global daily-reset/nutrition recovery requirement.

- Boss encounter every 5th floor.

- $\text{DungeonMultiplier} = 1 + 0.05 \times (\text{Floor} - 1)$.

- $\text{BossModifier} = 1.25$ when $\text{Floor} \% 5 = 0$; otherwise 1.00.

- $\text{EnemyHP} = \text{BaseHP} \times \text{DungeonMultiplier} \times \text{BossModifier}$.

- $\text{EnemyAttack} = \text{BaseAttack} \times \text{DungeonMultiplier} \times \text{BossModifier}$.

- $\text{FloorReward} = 100 + 25 \times (\text{Floor} - 1)$ Coins for a cleared normal floor.
- On a boss floor: $\text{BossReward} = \text{FloorReward} \times 3$.
- A floor only pays after it is cleared. Coins from previously cleared floors are retained even when the run later ends; there is no separate cash-out mechanic.

Dungeon UI / hackathon implementation note

- A dedicated Dungeon UI/assets do not need to be built from scratch for the hackathon. Reuse existing random monster/game assets already available in NutriQuest as the Dungeon enemy entities.
- Bosses can reuse those same existing assets: render the boss asset at a larger scale and add a red aura/glow around it so the boss is visually obvious without requiring a separate boss-art pipeline.
- This is a presentation/UI implementation shortcut only; Dungeon backend logic remains unchanged.

Presentation / technical math to preserve

- Rank thresholds: 100-RR bands from Iron through Platinum; Diamond begins at 500 RR.
- $\text{Adjustment} = \text{clamp}(\text{round}((\text{OpponentRR} - \text{PlayerRR}) / 25), -5, +5)$.
- $\text{WinRR} = 20 + \text{Adjustment}$; $\text{LossRR} = -15 + \text{Adjustment}$.
- MonsterPower, TeamPower, TeamGap%, and MatchScore SBMM formulas.
- Six rank-specific seven-rarity Case probability distributions.
- $\text{ExpectedRewardValue}(\text{rank}) = \sum [P(\text{case rarity} \mid \text{rank}) \times E(\text{NetWorth} \mid \text{rarity})]$.
- $\text{DungeonMultiplier} = 1 + 0.05 \times (\text{Floor} - 1)$; $\text{BossModifier} = 1.25$ every 5th floor.
- EnemyHP/EnemyAttack scaling formulas and $\text{FloorReward} = 100 + 25 \times (\text{Floor} - 1)$, with $3\times$ boss rewards.

6. PLAYER PROGRESSION

- No account XP or player-level system. Competitive progression is represented by Ranked RR/rank instead.
- Profile includes Battle History / Stats. Completed battles store result (win/loss), opponent, mode, date/time, RR change, and the 3 monsters used. Profile can show Wins, Losses, Win Rate, Current Rank/RR, with $\text{WinRate} = \text{Wins} / (\text{Wins} + \text{Losses}) \times 100$. Friendly battles can appear in history but do not affect Ranked RR/stats. Dungeon progression is tracked separately using HighestFloorReached.
- Daily tasks: 3 per day — 1 nutrition/scanning task, 1 gameplay/battle task, and 1 flexible task. Each completed task awards 250 Coins; completing all 3 awards an additional 500 Coins. Maximum normal task reward = $(3 \times 250) + 500 = 1,250$ Coins/day.
- Eligible tasks may additionally award +5 RR, capped at +10 task RR/day. Task RR cannot promote a player into the next rank. Eligible health/nutrition tasks may also satisfy early faint recovery.
- Nutrition streak: completing at least one eligible nutrition action per day maintains the streak. Every 5 consecutive streak days earns 1 stored Cookbook Boost. $\text{BoostsEarned} = \text{floor}(\text{StreakDays} / 5)$ for milestone tracking; a new boost is granted at each 5-day milestone. Using a boost does not reset the active streak.

- Cookbook Boost: one stored boost can be manually applied to one Cookbook opening and is consumed after use. Rare, Epic, Legendary, Mythic, and Secret rarity weights are multiplied by 1.15; Common and Uncommon retain their base weights. All seven weights are then renormalized to 100%: $\text{BoostedWeight}(r) = \text{BaseWeight}(r) \times 1.15$ for Rare+; $\text{FinalProbability}(r) = \text{BoostedWeight}(r) / \sum \text{BoostedWeights}$.

- UI / hackathon implementation note: there is currently NO dedicated UI for stored Cookbook Boosts and NO finalized visual asset/icon showing what a Cookbook Boost looks like. The backend logic can be implemented first. A simple placeholder indicator/icon may be used for the hackathon; a dedicated boost asset and polished profile/Cookbook boost UI can be designed later.

- Presentation math to preserve: WinRate formula; 1,250 Coins/day task maximum; +5 RR/task with +10/day cap; 5-day streak milestones; $\text{BoostsEarned} = \text{floor}(\text{StreakDays} / 5)$; Rare+ weight $\times 1.15$ followed by probability renormalization.

7. NUTRITION / HEALTH LAYER

- Personalized health profile is kept. During account creation, the user can enter weight, height, age, sex, activity level, and goal so NutriQuest can calculate BMI/BMR/TDEE and personalized daily calorie/macro targets.

- Signup / login UI note: NutriQuest still needs a dedicated Sign Up page and Login page. Signup should include the health-profile questions as part of onboarding rather than hiding them later in settings.

- Watch-data consent note: during signup/onboarding, ask whether the user wants to connect/share supported smartwatch health data with NutriQuest.

- Watch-enabled daily tasks: if the user opted in to supported smartwatch data, the daily-task pool may include activity-based goals such as reaching a step target or completing at least one recorded workout that day.

Nutrition target control: NutriQuest may calculate suggested calorie and macro targets from the user's profile, but the user remains in control.

Meal logging + home nutrition dashboard:

- Food can be logged by barcode scan, meal photo, or manual entry.
- Barcode scan pulls packaged-food nutrition and can also create the barcode monster.
- Meal photo estimates food, portion size, calories, protein, carbs, and fat; photo results should be labeled as estimates and never create a monster.
- Manual entry lets the user correct or enter nutrition values directly.
- Every logged meal can be edited or removed by the user.
- The main/home nutrition dashboard should show only kcal, protein, carbs, and fat progress against the user's chosen daily targets.

Nutrition-to-gameplay links (locked):

- Daily tasks may include nutrition actions such as logging a meal, scanning a barcode, reaching a chosen protein target, or staying within a user-defined calorie range.
- Selected eligible nutrition/health tasks may revive one fainted monster early instead of waiting for the next daily reset.

- Completing at least one eligible nutrition action can maintain the existing nutrition streak that awards a stored Cookbook Boost every 5 consecutive streak days.
- Do not directly increase or decrease combat stats based on whether the user hits calorie or macro goals. Nutrition adherence should not create combat buffs or penalties.
- Goal-based tasks should remain flexible and user-controlled; avoid requiring an exact calorie target when a user-defined range or logging-based task works instead.

- Fiber and water are not core home-page metrics. They may appear later only in a more detailed nutrition view if desired.

- Example calculation: $\text{RemainingCalories} = \text{DailyCalorieTarget} - \text{CaloriesConsumed}$. The same progress concept applies to protein, carbs, and fat.

The user can choose their own weight-loss, maintenance, or weight-gain approach, including how aggressive or conservative they want the calorie deficit/surplus to be. Daily targets such as calories, protein, carbohydrates, fat, fiber, and water should be user-modifiable rather than locked by the app. Suggested values are recommendations, not mandatory limits. These tasks should only appear when the relevant watch/health permission is connected and the required data is available.

- Users who do not connect watch data should receive equivalent non-watch tasks instead, so watch integration never becomes mandatory for daily rewards or progression.

This must be optional and opt-in; declining should not block account creation or normal gameplay. The exact watch-data fields and permissions will be finalized with the HealthKit/watch-data pipeline.

- Signup UI should clearly distinguish required account fields from optional health/watch-data permissions.

No comeback reward system. Returning after inactivity does not grant a free Cookbook, Case, monster, Coins, or other automatic catch-up reward.

8. WAGER / MINIGAME SYSTEM

- The casino/minigame system is already fully implemented in the existing NutriQuest codebase.

- Reuse the existing casino game UI and backend logic completely; do not redesign or rebuild these systems from scratch for the hackathon build.

- Reuse the current Cauldron Crash, Kitchen Mines, Plinko, and Portal Wheel implementations, including their wager handling, probability logic, settlement flow, nonce/idempotency behavior, ledger writes, and supporting backend logic.

- The existing codebase is the source of truth for System 8 game formulas, odds, and payout behavior. For presentation/documentation, extract the exact implemented math from the code rather than inventing a second design version.

- Any System 8 changes should be limited to integration, naming/theme polish, bug fixes, or compatibility with the finalized character/economy systems unless a specific redesign is intentionally requested.

IMPLEMENTATION GUARDRAILS

- characterMutations.ts — every monster/coin mutation in BEGIN IMMEDIATE , writes character_ledger + coin_ledger
- Inventory cap 200 unlocked rows; locked (escrowed) never evicted
- rewardFor mints $\leq$ budget, never above
- Rate limits in-memory per player: scan 10/min, battle 60/min, ranked+async 20/min, arena+dungeon 10/min, promos 5/min
- Zod on every body; no client-trusted stats anywhere post-audit
- Auth infrastructure can retain the existing script password hashing, 30-day bearer sessions, and human-gate logic unless intentionally changed during implementation.

CHECKLIST

This section contains only additional implementation tasks approved during the TODO audit. Items already fully covered by Systems 1–8 are not duplicated here. Rejected/removed tasks are not added.

☐ Disable/not implement player-to-player trading, gifting, marketplace transfers, direct player sales, and withdrawals. Monster selling remains system-only: selling a monster permanently removes it and pays its Current Net Worth in Coins.

[] Simplified food-scan anti-cheat: when a barcode monster is created, store the barcode, the nutrition snapshot actually used for generation, the nutrition-data source, and the creation timestamp. Reject or safely handle clearly missing/impossible/outlier nutrition values before minting. Do not build a complex canonical-product or source-confidence system for the hackathon.

[] Repeat-scan rule: each unique barcode may mint a monster only once per user. Later scans of the same barcode may still be used for meal/nutrition logging and may count for eligible logging/streak/task progress, but they never mint another monster or grant duplicate discovery rewards. Multiple scans of the same barcode on the same day may still log food when appropriate but do not create additional monster/reward farming.

[] Lightweight photo-scan validation: photo scans remain nutrition-only and never mint monsters. Generate the estimate normally; if model confidence is low or the calories/macros are clearly implausible, flag the result for confirmation and let the user edit it before saving to the nutrition log. Do not build duplicate-image hashing, photo mint limits, or a complex macro-reconciliation system for the hackathon.

[] Source-neutral combat balance: barcode-generated monsters and Cookbook/case-generated monsters use the same combat rules and power scale. Acquisition source must not provide any built-in combat advantage or disadvantage. If two monsters have the same character, Base Health, Base Attack, rarity, stars, moves, and Mana state, they perform identically in battle regardless of how they were obtained.

[] Create one master authored-character catalog for the 14 current character designs. Each entry should use a permanent character ID and include display name, base Health, base Attack, the 3 standard moves, Epic+ Special Ability, Base Mana where applicable, artwork/asset reference, and an optional short description/lore field. Do not include elements/types; NutriQuest has no type system. Only add rarity eligibility if a character is intentionally restricted from specific rarities.

[] Character IDs are immutable internal identifiers and must never be derived from display names. Renaming a character must not change references used by inventory, fusion, battle history, ownership, assets, or saved data.

[] Cookbook balance simulation: run a practical simulation of roughly 100,000–1,000,000 openings per Cookbook and compare observed rarity frequencies against the published odds. Also verify average monster Net Worth, observed sell-back percentage, and expected-value behavior. Do not include pity-system checks because NutriQuest has no pity system, and do not require a 10-million-run production-grade simulator for the hackathon.

[] Unbiased Cookbook/Case RNG: convert HMAC/hash output into the required probability range using an unbiased method. Do not use raw modulo ($\% N$) when it would introduce modulo bias, so observed long-run odds remain consistent with the published probabilities.

[] Atomic Cookbook opening: Coin debit $\rightarrow$ rarity RNG $\rightarrow$ Case/monster mint $\rightarrow$ ledger writes must succeed as one transaction or all fail together. Never allow a partial open that spends Coins without awarding the result, or awards a result without charging the Coins.

[] Battle-start squad snapshot: when a battle begins, snapshot both selected 3-monster teams and use those locked combat stats, moves, rarity/star scaling, Mana state, and other battle-relevant values for the entire match. Inventory changes, fusion, selling, renaming, or other monster updates made after the match begins must not alter the active battle.

[] Lightweight battle event log: store a simple server-side turn-by-turn log for completed matches, including turn number, player action, move or Special used, hit/miss, damage, critical hit, status/effect result, switches, and faints where applicable. Use it for battle history/debugging. Do not build battle seed commitments, rules-version tracking, final-state hashes, or a full cryptographic replay/verifier system for the hackathon.

[] Minimum-damage rule: if a damaging move successfully hits and the calculated FinalDamage would be below 1, deal 1 damage. Misses still deal 0 damage. Do not add a global maximum per-hit damage cap for the hackathon.

[] Ranked PvP anti-stall protection: use a per-turn player timer and a maximum total turn count so matches cannot continue indefinitely. If the maximum turn count is reached, resolve the match deterministically by comparing remaining non-fainted monsters first, then remaining total HP percentage if needed. Do not add a global anti-heal cap unless playtesting shows healing loops are an actual problem.

[] Ranked leaderboard ordering: sort players by RR first, then Ranked Wins as the first tiebreaker, then Win Rate as the second tiebreaker. Player level is not used because NutriQuest has no player-level system.

[] Lightweight fusion validation: before fusing, verify the 3 selected monsters are owned by the player, unlocked, not already being used in another pending mutation, and still match the required same-character + same-rarity + same-star combination. Consume the 3 inputs and create the fused result in one atomic transaction so a partial fusion cannot occur.

[] Inventory overflow protection: keep the existing 200-unlocked-monster inventory cap, but never silently delete or lose a newly earned reward because the inventory is full. If a reward would exceed the cap, place it in a simple overflow/mailbox state or otherwise hold the reward until the player creates space.

[] OPTIONAL — Monster provenance/history: later, consider storing a lightweight origin/history record for each monster, such as acquisition source and basic fusion ancestry. This is not required for the hackathon build and should not block core implementation.

[] Lightweight core-formula tests: add focused unit tests for the important finalized math and edge cases, including rarity odds, Net Worth/star bonuses, fusion requirements, damage calculations, RR changes, and Cookbook expected-value logic. Do not build a massive production-grade test suite for the hackathon.

[] Lightweight mutation/race-condition tests: add a small set of tests confirming concurrent or repeated requests cannot duplicate a monster, double-spend Coins, open the same reward twice, or fuse the same monster more than once. Focus on the core economy-changing paths rather than full production-scale concurrency testing.